

SST Builder and SST Iterator

 This is a legacy version of the Mini-LSM course and we will not maintain it anymore. We now have a better version of this course and this chapter is now part of [Mini-LSM Week 1 Day 4: Sorted String Table \(SST\)](#).

- [Task 1 - SST Builder](#)
- [Task 2 - SST Iterator](#)
- [Extra Tasks](#)

In this part, you will need to modify:

- `src/table/builder.rs`
- `src/table/iterator.rs`
- `src/table.rs`

You can use `cargo x copy-test day2` to copy our provided test cases to the starter code directory. After you have finished this part, use `cargo x scheck` to check the style and run all test cases. If you want to write your own test cases, write a new module `#[cfg(test)] mod user_tests { /* your test cases */ }` in `table.rs`. Remember to remove `#![allow(...)]` at the top of the modules you modified so that cargo clippy can actually check the styles.

Task 1 - SST Builder

SST is composed of data blocks and index blocks stored on the disk. Usually, data blocks are lazily loaded – they will not be loaded into the memory until a user requests it. Index blocks can also be loaded on-demand, but in this course, we make simple assumptions that all SST index blocks (meta blocks) can fit in memory. Generally, an SST file is of 256MB size.

The SST builder is similar to block builder – users will call `add` on the builder. You should maintain a `BlockBuilder` inside SST builder and split block when necessary. Also, you will need to maintain block metadata `BlockMeta`, which includes the first key in each block and the offset of each block. The `build` function will encode the SST, write everything to disk using `FileObject::create`, and return an `SsTable` object. Note that in part 2, you don't need to actually write the data to the disk. Just store everything in memory as a vector until we implement a block cache (Day 4, Task 5).

The encoding of SST is like:

Block Section		Meta Section	
Extra			
data block	... data block	meta block	... meta block meta block
offset (u32)			

You also need to implement `estimated_size` function of `SsTableBuilder`, so that the caller can know when can it start a new SST to write data. The function don't need to be very accurate. Given the assumption that data blocks contain much more data than meta block, we can simply return the size of data blocks for `estimated_size`.

You can also align blocks to 4KB boundary so as to make it possible to do direct I/O in the future. This is an optional optimization.

The recommend sequence to finish **Task 1** is as below:

- Implement `SsTableBuilder` in `src/table/builder.rs`
 - Before implementing `SsTableBuilder`, you may want to take a look in `src/table.rs`, for `FileObject` & `BlockMeta`.
 - For `FileObject`, you should at least implement `read`, `size` and `create` (No need for Disk I/O) before day 4.
 - For `BlockMeta`, you may want to add some extra fields when encoding / decoding the `BlockMeta` to / from a buffer.
- Implement `SsTable` in `src/table.rs`
 - Same as above, you do not need to worry about `BlockCache` until day 4.

After finishing **Task 1**, you should be able to pass all the current tests except two iterator tests.

Task 2 - SST Iterator

Like `BlockIterator`, you will need to implement an iterator over an SST. Note that you should load data on demand. For example, if your iterator is at block 1, it should not hold any other block content in memory until it reaches the next block.

`SsTableIterator` should implement the `StorageIterator` trait, so that it can be composed with other iterators in the future.

One thing to note is `seek_to_key` function. Basically, you will need to do binary search on block metadata to find which block might possibly contain the key. It is possible that the key

doesn't exist in the LSM tree so that the block iterator will be invalid immediately after a seek. For example,

```

-----
| block 1 | block 2 | block meta |
-----
| a, b, c | e, f, g | 1: a, 2: e |
-----

```

If we do `seek(b)` in this SST, it is quite simple – using binary search, we can know block 1 contains keys `a <= keys < e`. Therefore, we load block 1 and seek the block iterator to the corresponding position.

But if we do `seek(d)`, we will position to block 1, but seeking `d` in block 1 will reach the end of the block. Therefore, we should check if the iterator is invalid after the seek, and switch to the next block if necessary.

Extra Tasks

Here is a list of extra tasks you can do to make the block encoding more robust and efficient.

Note: Some test cases might not pass after implementing this part. You might need to write your own test cases.

- Implement index checksum. Verify checksum when decoding.
- Explore different SST encoding and layout. For example, in the [Lethe](#) paper, the author adds secondary key support to SST.

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/mini-lsm.

mini-lsm-book © 2022-2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.